

Universidad Rafael Landívar
Facultad de Ingeniería
Ciencia de Datos
Sección 2
Segundo Semestre 2026
Ing. Max Cerna

LABORATORIO 9

¿Importa cómo partimos los datos?

1. Introducción

Este laboratorio es **la precuela de la clase**, aun no vemos en clase las técnicas de partición de datos, ustedes van a observar el problema antes.

La regla del juego: el modelo de entrenamiento a utilizar será la regresión logística y como baseline un modelo Dummy, lo único que cambiará de una parte a otra es la forma de separar los datos en entrenamiento y prueba.

Todo lo que vean moverse en las métricas es consecuencia de esa decisión y de nada más.

En cada etapa van a **predecir** qué creen que va a pasar antes de ejecutar el código, **ejecutar** y anotar el resultado, y **comparar** su predicción con lo que ocurrió.

Si la predicción falla, no se borra o ignora, esa diferencia la necesitamos discutir en la clase.

Lo mínimo que necesitamos para arrancar

- **Train:** los datos con los que el modelo aprende.
- **Test:** datos que el modelo no vio, con los que medimos qué tan bien le fue.
- **Métrica:** usamos **F1** para clasificación (combina precisión y recall) y **MAE** para regresión (error promedio en minutos). Entre más alto el F1, mejor, entre más bajo el MAE, mejor.

2. Objetivos del laboratorio

- **Medir:** comprobar cuánto se mueve el desempeño de un mismo modelo cuando lo único que cambia es la forma de partir los datos.
- **Comparar:** contrastar cinco maneras distintas de separar train y test sobre el mismo set de datos y el mismo modelo.
- **Detectar:** reconocer cuándo una medición salió demasiado optimista, y explicar de dónde vino ese optimismo.



- **Decidir:** elegir y justificar, para un escenario de negocio concreto, qué número le reportaría al cliente.
- **Documentar:** dejar por escrito la predicción previa y la explicación posterior de cada experimento.

3. Roles

El notebook es **uno solo por trío**, pero cada integrante debe poder explicar cualquier parte

Rol	Responsabilidad
Quien programa	Escribe y ejecuta el código
Quien registra	Llena las tablas de resultados y las predicciones
Quien cuestiona	Antes de cada ejecución obliga al trío a predecir, y anota los desacuerdos

4. Preparación

Paso 1 – Cargar

```
import pandas as pd, numpy as np

df = pd.read_csv('pedidos_ruta_verde.csv', parse_dates=['fecha'])
df.info()
df.isna().sum()
```

Paso 2 – Reconocer el set de datos

Contesten en una celda de texto:

- ¿Cuántos pedidos hay, cuántos restaurantes distintos y qué rango de fechas cubre el archivo?
- ¿Qué columnas tienen nulos y cuántos?
- Revisen además el mínimo y el máximo de **tiempo_entrega_min**: ¿hay valores imposibles?

Paso 3 – Definir las dos preguntas de negocio

```
# Pregunta 1 (clasificación): ¿este pedido recibirá calificación baja?
cal = df.dropna(subset=['calificacion']).copy()
cal['baja'] = (cal['calificacion'] <= 2).astype(int)
print(cal['baja'].mean())           # ¿qué porcentaje de pedidos es "baja"?

# Pregunta 2 (regresión): ¿cuántos minutos tardará el pedido?
reg = df[(df.tiempo_entrega_min > 0) & (df.tiempo_entrega_min < 200)].copy()
```

Pregunta 1. Si quieren predecir el tiempo de entrega en el momento en que entra el pedido, ¿cuál de las columnas del set de datos no podrían usar como insumo? Justifiquen preguntándose, para cada columna: ¿este dato ya existe cuando el pedido entra?

5. Definición del modelo

Copien los dos bloques siguientes tal cual.

Paso 1 – M1 (regresión logística)

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score, recall_score

NUM = ['distancia_km', 'total_pedido_gtq']
CAT = ['zona', 'categoria_restaurante', 'metodo_pago']
FEATURES = NUM + CAT

def nuevo_modelo():
    """M1. Siempre se crea nuevo; nunca se reutiliza uno ya entrenado."""
    prep = ColumnTransformer([
        ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                          ('esc', StandardScaler())]), NUM),
        ('cat', OneHotEncoder(handle_unknown='ignore'), CAT),
    ])
    return Pipeline([
        ('prep', prep),
        ('clf', LogisticRegression(max_iter=1000, class_weight='balanced')),
    ])

def entrenar_y_medir(train, test, target='baja'):
```

```
"""Entrena M1 con 'train', mide en 'test' y devuelve F1 y recall."""
m = nuevo_modelo()
m.fit(train[FEATURES], train[target])
pred = m.predict(test[FEATURES])
return {'f1': f1_score(test[target], pred),
        'recall': recall_score(test[target], pred),
        'n_test': len(test),
        'positivos_test': int(test[target].sum())}
```

Observación: M1 es una regresión logística a la que antes se le rellenan los nulos con la mediana, se le escalan las variables numéricas y se le convierten las categóricas en columnas 0/1.

Paso 2 – M0 (El Baseline “tonto”)

```
def medir_baseline(train, test, target='baja'):
    """M0: modelo "tonto". Predice siempre la clase más común."""
    d = DummyClassifier(strategy='most_frequent')
    d.fit(train[FEATURES], train[target])
    return f1_score(test[target], d.predict(test[FEATURES]))
```

Regla del laboratorio: cada vez que reporten un F1 de M1, reporten al lado el de M0.

Pregunta 2. Antes de ejecutar nada: ¿qué F1 creen que va a sacar M0, el modelo que siempre responde “no será baja”? Anoten su predicción.

6. Parte A – Cinco resultados con la misma base

Vamos a separar los datos al azar: 80 % para entrenar, 20 % para probar.

El azar se controla con un número llamado **random_state**, cambiarlo reparte los pedidos de otra forma.

Predicción del grupo (anótenla antes de ejecutar): si solo cambia ese número y el modelo es idéntico, ¿el F1 va a ser igual, parecido o muy distinto?



```
from sklearn.model_selection import train_test_split

filas = []
for semilla in [0, 1, 7, 42, 2026]:
    train, test = train_test_split(cal, test_size=0.2, random_state=semilla)
    r = entrenar_y_medir(train, test)
    r['semilla'] = semilla
    r['f1_baseline'] = medir_baseline(train, test)
    filas.append(r)

pd.DataFrame(filas)
```

Llenen esta tabla en el notebook:

Semilla	Positivos en test	% positivos	F1 de M1	F1 de M0
0				
1				
7				
42				
2026				
mín - máx				

Pregunta 3. ¿Cuánta diferencia hay entre el mejor y el peor F1? ¿Les parece poco o mucho?

Pregunta 4. Hablen con otro grupo, si otro grupo reporta un F1 distinto al suyo con el mismo modelo y el mismo set de datos, ¿quién tiene razón? ¿Qué número debería reportarse entonces?

Pregunta 5. Miren la columna de positivos en el test. ¿Qué relación notan entre esa columna y el F1?

7. Parte B - Forzar la proporción

En `train_test_split` existe un parámetro llamado **stratify**. Esto es lo único que les decimos: obliga a que el train y el test tengan la misma proporción de cada clase que el set completo.

Predicción del grupo: con `stratify`, ¿el F1 promedio va a subir, o va a cambiar otra cosa?



```
train, test = train_test_split(cal, test_size=0.2, random_state=semilla,  
                              stratify=cal['baja'])
```

Repitan la tabla de la Parte A con las mismas cinco semillas.

Pregunta 6. Comparen las dos tablas. ¿Qué cambió: el promedio del F1, la diferencia entre semillas, o ambos?

Pregunta 7. Entonces, ¿stratify hace que el modelo prediga mejor, o hace otra cosa? Expliquen con sus palabras qué creen que hace.

Pregunta 8. ¿En qué tipo de problema creen que stratify sería indispensable? ¿Y en cuál daría igual usarlo o no?

8. Parte C – Separar por fecha

Hasta ahora el azar mezcló pedidos de todos los meses. Ahora vamos a partir por tiempo.

Paso 1 – Mirar el comportamiento mes a mes

Calculen y grafiquen el porcentaje de pedidos con calificación baja mes por mes. Observen bien el resultado antes de seguir.

Paso 2 – Entrenar con el pasado y probar con el futuro

Predicción del grupo: si entrenan con enero-junio y prueban con julio-agosto, ¿el F1 va a salir mejor, igual o peor que con el split aleatorio?

```
train_t = cal[cal.fecha < '2026-07-01']  
test_t = cal[cal.fecha >= '2026-07-01']  
entrenar_y_medir(train_t, test_t), medir_baseline(train_t, test_t)
```

Pregunta 9. ¿Acertaron en su predicción? ¿Qué explicación tienen para el resultado?

Pregunta 10. Ruta Verde quiere usar el modelo en septiembre, con pedidos que todavía no existen. De las dos mediciones que ya tienen (aleatoria y por fecha), ¿cuál se parece más a esa situación? ¿Por qué?

Pregunta 11. El split aleatorio permite que un pedido del 8 de agosto esté en el train mientras uno del 3 de febrero está en el test. ¿Qué tiene de raro eso si pensamos en cómo se usará el modelo en la vida real?



9. Parte D – Separar por restaurante

Ruta Verde trabaja con 7 restaurantes, cada uno con entre 71 y 98 pedidos.

Paso 1 – ¿Quién aparece en los dos lados?

Con cualquiera de los splits aleatorios anteriores, cuenten cuántos de los restaurantes que aparecen en el test aparecen también en el train.

Paso 2 – Dejar dos restaurantes completos fuera del entrenamiento

Predicción del grupo: si quitamos del entrenamiento dos restaurantes completos y probamos solo con ellos, ¿qué le pasará al F1?

```
from sklearn.model_selection import GroupShuffleSplit

gss = GroupShuffleSplit(n_splits=1, test_size=2/7, random_state=42)
i_tr, i_te = next(gss.split(cal, cal['baja'], groups=cal['restaurante']))
train_g, test_g = cal.iloc[i_tr], cal.iloc[i_te]
print(train_g.restaurante.unique(), test_g.restaurante.unique())
entrenar_y_medir(train_g, test_g), medir_baseline(train_g, test_g)
```

Pregunta 12. ¿Qué pasó con el F1 y por qué creen que pasó?

Pregunta 13. Mañana Ruta Verde firma con un restaurante nuevo, que nunca ha tenido pedidos. De todas las mediciones del laboratorio, ¿cuál describe mejor cómo se va a portar el modelo con ese restaurante?

10. Parte E – Un atajo tentador

Una idea que suena razonable, como el restaurante parece influir, agreguemos una columna con el promedio histórico de calificaciones bajas de cada restaurante y usémosla como insumo del modelo.

Paso 1 – Versión 1: calcular el promedio con todo y después partir

```
prom = cal.groupby('restaurante')['baja'].mean() # usa TODOS los pedidos
cal_v1 = cal.assign(riesgo_rest=cal['restaurante'].map(prom))
# luego: train_test_split(cal_v1, ...) y medir
# (recuerden agregar 'riesgo_rest' a la lista NUM)
```

Paso 2 – Versión 2: partir primero y calcular el promedio solo con el train

```
train, test = train_test_split(cal, test_size=0.2, random_state=42,  
                              stratify=cal['baja'])  
prom_tr = train.groupby('restaurante')['baja'].mean() # usa SOLO el train  
train_v2 = train.assign(riesgo_rest=train['restaurante'].map(prom_tr))  
test_v2 = test.assign(riesgo_rest=test['restaurante'].map(prom_tr))
```

Predicción del grupo: ¿cuál de las dos versiones va a dar mejor F1?

Pregunta 14. ¿Cuál dio mejor? ¿Qué información tenía disponible la versión 1 que la versión 2 no tenía?

Pregunta 15. Si entregan la versión 1 como resultado oficial y el modelo se pone en producción, ¿qué creen que va a pasar con pedidos reales? ¿Por qué?

Pregunta 16. ¿Se les ocurre otro cálculo del laboratorio que tenga el mismo problema si se hace antes de partir? Pista: miren qué hace SimpleImputer y qué hace StandardScaler dentro de M1.

11. Parte F – Su recomendación

Con todo lo que midieron, el ustedes deciden:

- Qué número le reportarían al gerente de Ruta Verde como desempeño esperado del modelo, y con qué forma de partir se obtuvo.
- Qué forma de partir usarían en cada uno de estos tres escenarios: (a) el modelo correrá cada noche sobre los pedidos del día siguiente; (b) Ruta Verde quiere saber si el modelo sirve para restaurantes nuevos; (c) solo se quiere comparar dos modelos entre sí, con los datos actuales.

Pregunta 17. ¿Cuál de todas las mediciones del laboratorio fue la más alta? ¿Y la más confiable? ¿Son la misma? Expliquen.



12. Tabla comparativa

Este es uno de los entregables. Llenen una fila por experimento:

Experimento	F1 de M1	F1 de M0	¿Qué situación real simula?
A · Aleatorio, mejor semilla			
A · Aleatorio, peor semilla			
B · Con stratify			
C · Por fecha (jul-ago)			
D · Por restaurante			
E · Versión 1 (promedio con todo)			
E · Versión 2 (promedio con train)			

13. Para llevar a la clase

Esto es lo que vamos a discutir en la siguiente clase. En media página escriban:

- El hallazgo que más les sorprendió, y por qué.
- Una pregunta sin responder que les quedó del laboratorio.
- Una regla que ustedes propondrían para el equipo de Ruta Verde, escrita en una sola frase y sin tecnicismos.

14. Entregables y evaluación

Entregable	Puntos
Notebook con las partes A-F, incluyendo las predicciones anotadas antes de ejecutar	35
Respuestas a las 17 preguntas, en celdas de texto	20
Tabla comparativa (punto 12)	10
Para llevar a la clase (punto 13)	5
Defensa oral: el docente elige a quién le pregunta cada cosa	30

Criterio de calificación: se evalúa el razonamiento, no el valor de la métrica. Una predicción equivocada seguida de una buena explicación vale más que un resultado correcto sin justificación.



15. Reglas de entrega

- Un notebook (.ipynb) por grupo, con los tres nombres y el rol que tuvo cada quien.
- El bloque del modelo no se modifica. Si lo cambian, los resultados dejan de ser comparables entre grupos.
- Las celdas de texto son obligatorias: un notebook con puro código no se califica.
- Si usan alguna herramienta de IA, indiquen cómo la utilizaron y expliquen qué les aportó. En la defensa oral deben poder reconstruir el razonamiento sin ella.

